

KLINIEK VAN DIJK — FRONTEND COURSE

Reactivity in Svelte, step by step

From local component state to shared state and context

Steps

Step 1 — Local component state

The simplest way to work with state in Svelte: the state lives in the same component as everything that reads and writes it. No props, no lifting state up, no separate store.

- `selectedVet` is declared with `$state` right next to the `vets` prop.
- Clicking a vet card writes to it directly with `onClick={() => (selectedVet = vet)}`.
- The summary panel reads `selectedVet` in an `{#if}` — Svelte re-renders it automatically whenever `selectedVet` changes, because it's the same component.

`src/lib/components/organisms/BookingFormSection.svelte`:

```
<script lang="ts">
  import type { Vet } from '../..data';
  import VetCard from '../molecules/VetCard.svelte';

  interface Props {
    vets: Vet[];
  }

  let { vets }: Props = $props();

  let selectedVet = $state<Vet | null>(null);
</script>

<section class="mx-auto max-w-6xl px-4 py-16">
  <h1 class="text-3xl font-bold text-primary-900">Book an appointment</h1>
  <p class="mt-3 text-slate-600">Choose which vet you'd like to see.</p>

  <div class="mt-10 grid gap-8 md:grid-cols-3">
    <div class="grid gap-6 sm:grid-cols-2 md:col-span-2">
      {#each vets as vet}
        <button type="button" class="text-left" onClick={() => (selectedVet = vet)}>
          <VetCard {vet} />
        </button>
      {/each}
    </div>
  </div>
```

```
<div class="h-fit rounded-2xl border border-slate-200 bg-slate-50 p-6 shadow-sm
md:col-span-1">
  <h2 class="text-lg font-semibold text-primary-900">Summary</h2>

  {#if selectedVet}
    <p class="mt-4 text-sm text-slate-700">{selectedVet.name}</p>
  {:else}
    <p class="mt-4 text-sm text-slate-500">No vet selected yet.</p>
  {/if}
</div>
</div>
</section>
```

Step 2 — Extract a presentational component, pass data down via props

The summary UI becomes its own component. State still lives where the click happens (`BookingFormSection.svelte`) — the new component only receives data through a prop and renders it. This introduces one-directional data flow: state owned by the parent, read-only data passed down to a child.

- New file: `src/lib/components/molecules/AppointmentSummary.svelte`.
- It declares one prop, `selectedVet: Vet | null`, and has no state of its own.
- `BookingFormSection.svelte` keeps `selectedVet = $state<Vet | null>(null)` and the `onClick` handlers, and renders `<AppointmentSummary {selectedVet} />` instead of the inline markup.

`src/lib/components/molecules/AppointmentSummary.svelte:`

```
<script lang="ts">
  import type { Vet } from '../..data';

  interface Props {
    selectedVet: Vet | null;
  }

  let { selectedVet }: Props = $props();
</script>

<h2 class="text-lg font-semibold text-primary-900">Summary</h2>

{#if selectedVet}
  <p class="mt-4 text-sm text-slate-700">{selectedVet.name}</p>
{:else}
  <p class="mt-4 text-sm text-slate-500">No vet selected yet.</p>
{/if}
```

`src/lib/components/organisms/BookingFormSection.svelte:`

```
<script lang="ts">
  import type { Vet } from '../..data';
  import VetCard from '../molecules/VetCard.svelte';
```

```

import AppointmentSummary from '../molecules/AppointmentSummary.svelte';

interface Props {
  vets: Vet[];
}

let { vets }: Props = $props();

let selectedVet = $state<Vet | null>(null);
</script>

<section class="mx-auto max-w-6xl px-4 py-16">
  <h1 class="text-3xl font-bold text-primary-900">Book an appointment</h1>
  <p class="mt-3 text-slate-600">Choose which vet you'd like to see.</p>

  <div class="mt-10 grid gap-8 md:grid-cols-3">
    <div class="grid gap-6 sm:grid-cols-2 md:col-span-2">
      {#each vets as vet}
        <button type="button" class="text-left" onclick={() => (selectedVet = vet)}>
          <VetCard {vet} />
        </button>
      {/each}
    </div>

    <div class="h-fit rounded-2xl border border-slate-200 bg-slate-50 p-6 shadow-sm md:col-span-1">
      <AppointmentSummary {selectedVet} />
    </div>
  </div>
</section>

```

Step 3 — Shared state

Instead of passing `selectedVet` down through props, both components import the same reactive object from a `.svelte.ts` module. Neither has to know about the other, and no data flows between them through the component tree — they just both point at the same state.

- New file: `src/lib/state/booking-state.svelte.ts`. A `.svelte.ts` file can use runes outside of a `.svelte` component.
- It exports `bookingState`, an object created with `$state`. Consumers mutate a property on it (`bookingState.selectedVet = vet`) rather than reassigning the imported name — you can't reassign an import from outside its module, but you can mutate an object it points to.
- `AppointmentSummary.svelte` no longer takes a `selectedVet` prop — it imports `bookingState` and reads `bookingState.selectedVet` directly.
- `BookingFormSection.svelte` drops its local `let selectedVet = $state(...)` entirely. The `onclick` now writes to `bookingState.selectedVet` instead, and `<AppointmentSummary />` is rendered with no props at all.

Note: the `onclick` stays on the wrapping `<button>` in `BookingFormSection.svelte`'s `{#each}` loop — it does not move into `VetCard.svelte`. `VetCard` is also used by `TeamSection` on the homepage, purely to display a vet. If `VetCard` imported `bookingState` directly, clicking a vet card on the homepage would silently mutate booking-page state too — that's a real bug, not just a style choice. Keeping the click handler in the booking-specific parent keeps `VetCard` reusable and unaware of booking.

`src/lib/state/booking-state.svelte.ts:`

```
import type { Vet } from '../data';

// A `.svelte.ts` file can use runes outside of a component. Any component
// that imports `bookingState` reads/writes the same object — no props
// needed to pass the selection between them.
export const bookingState = $state<{ selectedVet: Vet | null }>({ selectedVet: null });
```

`src/lib/components/molecules/AppointmentSummary.svelte:`

```
<script lang="ts">
  import { bookingState } from '../../state/booking-state.svelte';
</script>
```

```

<h2 class="text-lg font-semibold text-primary-900">Summary</h2>

{#if bookingState.selectedVet}
  <p class="mt-4 text-sm text-slate-700">{bookingState.selectedVet.name}</p>
{:else}
  <p class="mt-4 text-sm text-slate-500">No vet selected yet.</p>
{/if}

```

src/lib/components/organisms/BookingFormSection.svelte:

```

<script lang="ts">
  import type { Vet } from '../..data';
  import VetCard from '../molecules/VetCard.svelte';
  import AppointmentSummary from '../molecules/AppointmentSummary.svelte';
  import { bookingState } from '../..state/booking-state.svelte';

  interface Props {
    vets: Vet[];
  }

  let { vets }: Props = $props();
</script>

<section class="mx-auto max-w-6xl px-4 py-16">
  <h1 class="text-3xl font-bold text-primary-900">Book an appointment</h1>
  <p class="mt-3 text-slate-600">Choose which vet you'd like to see.</p>

  <div class="mt-10 grid gap-8 md:grid-cols-3">
    <div class="grid gap-6 sm:grid-cols-2 md:col-span-2">
      {#each vets as vet}
        <button type="button" class="text-left" onclick={() =>
(bookingState.selectedVet = vet)}>
          <VetCard {vet} />
        </button>
      {/each}
    </div>

    <div class="h-fit rounded-2xl border border-slate-200 bg-slate-50 p-6 shadow-sm
md:col-span-1">
      <AppointmentSummary />
    </div>
  </div>

```

```
    </div>  
  </div>  
</section>
```

Step 4 — Context instead of shared state

Step 3's `bookingState` was a singleton: one object, created once when the module first loads, shared by every component that imports it. That's fine here, but it hides a bug — if `BookingFormSection` were ever rendered twice on the same page, both instances would share the exact same `selectedVet`, and selecting a vet in one would also update the other.

Context fixes this by creating a fresh state object each time the component tree is built, and scoping it to that one subtree:

- `createBookingState()` no longer creates the object once at module load — it's a factory function that returns a new `$state` object every time it's called. `$state` still works here because the rune only needs to run during a component's initialization, which it does since a component's `<script>` calls this function.
- `setBookingContext` / `getBookingContext` wrap Svelte's `setContext` / `getContext` with a shared key and a return type, so callers don't repeat a raw string key or a type cast.
- `BookingFormSection.svelte` creates its own `bookingState` and calls `setBookingContext(...)` in its `<script>` — this makes that state available to everything rendered inside it.
- `AppointmentSummary.svelte` calls `getBookingContext()` instead of importing a shared object. This only works because it's rendered inside `BookingFormSection` — `getContext` looks up the tree for the nearest ancestor that called `setContext` with that key.

One gotcha worth calling out: `$state(...)` can only appear as a variable declaration initializer (`const state = $state(...)`) — `return $state(...)` directly is a compile error.

`src/lib/state/booking-state.svelte.ts`:

```
import { getContext, setContext } from 'svelte';
import type { Vet } from '../data';

interface BookingState {
  selectedVet: Vet | null;
}

const bookingContextKey = 'booking';

// A `.svelte.ts` file can use runes outside of a component, including
// inside a plain function like this one — `$state` just needs to run
```

```
// during a component's initialization, which it still does here since
// `createBookingState` is called from a component's <script>.
export function createBookingState(): BookingState {
  const state = $state<BookingState>({ selectedVet: null });
  return state;
}

export function setBookingContext(state: BookingState) {
  setContext(bookingContextKey, state);
}

export function getBookingContext(): BookingState {
  return getContext(bookingContextKey);
}
```

src/lib/components/molecules/AppointmentSummary.svelte:

```
<script lang="ts">
  import { getBookingContext } from '../..state/booking-state.svelte';

  const bookingState = getBookingContext();
</script>

<h2 class="text-lg font-semibold text-primary-900">Summary</h2>

{#if bookingState.selectedVet}
  <p class="mt-4 text-sm text-slate-700">{bookingState.selectedVet.name}</p>
{:else}
  <p class="mt-4 text-sm text-slate-500">No vet selected yet.</p>
{/if}
```

src/lib/components/organisms/BookingFormSection.svelte:

```
<script lang="ts">
  import type { Vet } from '../..data';
  import VetCard from '../molecules/VetCard.svelte';
  import AppointmentSummary from '../molecules/AppointmentSummary.svelte';
  import { createBookingState, setBookingContext } from '../..state/booking-
state.svelte';
```

```

interface Props {
  vets: Vet[];
}

let { vets }: Props = $props();

const bookingState = createBookingState();
setBookingContext(bookingState);
</script>

<section class="mx-auto max-w-6xl px-4 py-16">
  <h1 class="text-3xl font-bold text-primary-900">Book an appointment</h1>
  <p class="mt-3 text-slate-600">Choose which vet you'd like to see.</p>

  <div class="mt-10 grid gap-8 md:grid-cols-3">
    <div class="grid gap-6 sm:grid-cols-2 md:col-span-2">
      {#each vets as vet}
        <button type="button" class="text-left" onclick={() =>
(bookingState.selectedVet = vet)}>
          <VetCard {vet} />
        </button>
      {/each}
    </div>

    <div class="h-fit rounded-2xl border border-slate-200 bg-slate-50 p-6 shadow-sm
md:col-span-1">
      <AppointmentSummary />
    </div>
  </div>
</section>

```